

Strings, Packages, and Interfaces

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor(CSE), PIT
Parul University

1.	Enumerations in Java.....	1
2.	Enum Constructors, Methods, and values().....	2
3.	Comparing Enums vs. Constants.....	4
4.	Summary of Key Concepts.....	4
5.	References.....	4

7.3.1 Enumerations in Java

◆ What is an Enumeration?

An **enumeration** (enum) in Java is a special data type that enables a variable to be a set of predefined constants. It's used when a variable (especially of type int or String) should only hold a specific set of values.

◆ Defining an enum

You can define an enum using the enum keyword. It can be defined **outside** or **inside** a class (but not within methods):

```
enum Day {  
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY  
}
```

Each identifier in the enum list is a **constant object** of the Day type.

◆ Using an enum

Once an enum is defined, you can create variables of that type and assign them one of its constants:

```
public class TestEnum {  
    Day today = Day.MONDAY;  
  
    public void checkDay() {  
        if (today == Day.MONDAY) {  
            System.out.println("Start of the week!");  
        }  
    }  
}
```

◆ Benefits of Enums in Java

- **Type-safe:** Prevents invalid values.
- **Readable:** Constants are self-explanatory.
- **Grouped:** All related constants are in one place.
- **Extendable:** Enums can have constructors, methods, and fields.

 Example:

```
enum Level {  
    LOW, MEDIUM, HIGH  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Level myLevel = Level.HIGH;  
        System.out.println("Level is: " + myLevel);  
    }  
}
```

7.3.2 Enum Constructors, Methods, and values()

1. Enum Constructors

Enums in Java can have constructors, but they must be **private** or **package-private**. The constructor is called automatically with each constant declaration.

 Example:

```
enum Planet {  
    MERCURY(3.303e+23), VENUS(4.869e+24), EARTH(5.976e+24);  
  
    private final double mass; // in kilograms  
  
    // Constructor  
    Planet(double mass) {  
        this.mass = mass;  
    }  
  
    public double getMass() {  
        return mass;  
    }  
}
```

Here, each enum constant (e.g., MERCURY) invokes the constructor with a value.

2. Enum Methods

Enums can have methods like any class, including:

- **User-defined methods** (e.g., getMass() above).
- **Predefined methods** from java.lang.Enum, such as:

Method	Description
name()	Returns the name of the constant as a string.
ordinal()	Returns the index (position) of the enum constant (starting from 0).
valueOf(String name)	Converts a string to the corresponding enum constant. Throws <code>IllegalArgumentException</code> if the name is invalid.

Example:

```
public class TestEnumMethods {
    public static void main(String[] args) {
        Planet p = Planet.EARTH;
        System.out.println("Name: " + p.name());
        System.out.println("Index: " + p.ordinal());
        System.out.println("Mass: " + p.getMass());
    }
}
```

3. values() Method

This method returns an array of all the enum constants in the order they are declared. It is automatically generated by the compiler.

Example:

```
for (Planet p : Planet.values()) {
    System.out.println(p + " mass is " + p.getMass());
}
```

Enums with switch-case

Enums can be used directly in switch statements:

```
enum TrafficLight { RED, YELLOW, GREEN }

public class Test {
    public static void main(String[] args) {
        TrafficLight light = TrafficLight.RED;
        switch (light) {
            case RED:
                System.out.println("Stop!");
                break;
            case GREEN:
                System.out.println("Go!");
                break;
        }
    }
}
```

7.3.3 Comparing Enums vs. Constants

Feature	enum	final static constants
Type safety	✓ Yes	✗ No
Grouping	✓ Logical group	✗ No
Namespace	✓ Belongs to a type	✗ Global
Method support	✓ Can have methods and constructors	✗ Cannot have behavior
Use in switch-case	✓ Yes	✓ Yes (only primitives)

➤ Summary of Key Concepts:

- Enums in Java are special classes used to define a collection of constants.
- Each constant in an enum is a public static final object.
- Enums can have:
 - Constructors (only private/package-private),
 - Fields, and
 - Methods (user-defined and built-in).
- Common built-in methods:
 - `name()`: Returns the constant name.
 - `ordinal()`: Returns the position of the constant.
 - `values()`: Returns an array of all enum constants.
- Enums support switch-case statements.
- Compared to constants (final static), enums are type-safe, self-contained, and more expressive.

➤ References:

- Oracle Java Documentation
 <https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html>
- GeeksforGeeks Enums in Java
 <https://www.geeksforgeeks.org/enum-in-java/>
- Baeldung Java Enums
 <https://www.baeldung.com/java-enum>

Parul[®] University | **NAAC** **A++**
Vadodara, Gujarat | **GRADE**



    
<https://paruluniversity.ac.in/>